



École Nationale Supérieure de Techniques Avancées - ENSTA Paris

Problème du sac à dos multidimensionnel

Christopher Tsa-Kwet-Shin, Edouard Rimbault

Palaiseau

Janvier 2025

Table des matières

1	Présentation du problème du sac à dos multidimensionnel	2
2	Recherche de solutions réalisables par des heuristiques préliminaires	2
2.1	Structure de l'heuristique gloutonne	2
2.2	Structure de l'heuristique de réparation	3
3	Structures de voisinages	4
3.1	Construction des structures de voisinages	4
3.2	Notion de structures multiples de voisinages	4
3.3	Structures multiples de voisinages utilisées	4
4	Heuristique d'amélioration locale	10
4.1	Pseudo-code	10
4.2	Implémentation sur Python	10
5	Métaheuristiques	11
5.1	Métaheuristique de recherche à voisinages variables	11
5.2	Algorithme génétique	12
5.2.1	Étapes de l'algorithme génétique	13
5.2.2	Pseudo-code de l'algorithme génétique	14
5.3	Recuit simulé	14
5.3.1	Pseudo-code de l'algorithme génétique	15
5.4	Algorithme hybride SA-IGA	16
6	Exploitation des résultats	16
6.1	Structure des instances and métriques d'évaluation choisies	16
6.1.1	Description des instances de test	16
6.1.2	Métriques d'évaluation	17
6.2	Performances des heuristiques préliminaires	17
6.2.1	Performances globales	17
6.2.2	Sensibilité par rapport à N et M	17
6.2.3	Sensibilité par rapport à N et M	18
6.3	Performances de l'heuristique d'amélioration locale	19
6.3.1	Performances avec un voisinage 3-opt	19
6.4	Performances de l'algorithme génétique	20
7	Conclusion	22

1 Présentation du problème du sac à dos multidimensionnel

On s'intéresse à un problème d'école en optimisation combinatoire : Problème du sac à dos multidimensionnel (Multidimensional Knapsack Problem) .

Ce problème peut servir à modéliser le problème d'allocation de ressources (capital budgeting).

Étant donnés :

- N projets et M ressources
- b_i , Ya quantité de ressource i disponible ($i = 1, \dots, M$)
- c_j , le profit (ou gain) associé au projet j ($j = 1, \dots, N$)
- a_{ij} , la quantité de ressource i consommée par le projet j ($i = 1, \dots, M$ et $j = 1, \dots, N$)

Le problème d'allocation de ressources consiste à sélectionner un sous-ensemble de projets tel que le gain total soit maximisé dans la limite des ressources disponibles.

Une formulation classique de ce problème est la suivante :

$$\begin{array}{ll} \max & f(x) = \sum_{j=1}^N c_j x_j \\ \text{s.c.} & \sum_{j=1}^N a_{ij} x_j \leq b_i \quad i = 1, \dots, M \\ & x_j \in \{0, 1\} \quad j = 1, \dots, N \end{array}$$

avec $x_j = 1$ si le projet j est accepté ($x_j = 0$ sinon)

Hypothèses sur les données :

- c_j et b_i sont des entiers positifs
- $A^j = (a_{ij})_{i=1, \dots, M}$ est différent du vecteur nul $\forall j \in \{1, \dots, N\}$

L'objectif de ce devoir est la résolution approchée du problème de sac à dos multidimensionnel par l'intermédiaire d'une métaheuristique.

2 Recherche de solutions réalisables par des heuristiques préliminaires

Dans un premier temps, nous avons généré des solutions réalisables à l'aide de deux heuristiques : une heuristique gloutonne et une heuristique de réparation. L'objectif est trouver des solutions avec un profit suffisamment acceptable pour les utiliser comme solution initiale dans une heuristique de recherche locale ou une métaheuristique.

2.1 Structure de l'heuristique gloutonne

Nous avons implementé une heuristique gloutonne. L'algorithme de l'heuristique gloutonne se compose des étapes suivantes :

- Pour chaque projet j , on calcule le rapport profit/coût de la manière suivante :

$$\text{profit/coût}(j) = \frac{c_j}{\sum_{i=1}^M a_{ij}}$$

- On classe les projets dans l'ordre décroissant selon le rapport profit/coût.
- On parcourt les objets dans l'ordre du tri et on vérifie que : $\forall j, a_{ij} \leq b_j$
- Si la vérification est faite avec succès on ajoute le projet j dans la solution et on met à jour la capacité de toutes les ressources : $b_i \leftarrow b_i - a_{ij}$
- L'algorithme se termine quand tous les projets ont été traités.

L'algorithme est initialisé par un vecteur de 0 de taille N . Il a une complexité en $O(N \log N + N \cdot M)$.

2.2 Structure de l'heuristique de réparation

L'heuristique de réparation consiste à partir d'une solution non valide et modifier cette solution de manière à ce que toutes les contraintes violées deviennent valides. Nous avons choisi de générer la solution initiale via une méthode de relaxation substitutive. Cette méthode consiste à transformer les M contraintes en une seule contrainte avec des multiplicateurs λ_i de la manière suivante :

$$\sum_{j=1}^N \left(\sum_{i=1}^M \lambda_i a_{ij} \right) x_j \leq \sum_{i=1}^M \lambda_i b_i$$

Nous avons déterminé les λ_i en utilisant la méthode de tension. Nous avons préféré cette méthode à des méthodes itératives comme la descente de gradient ou la relaxation lagrangienne par souci de praticité. Ainsi, à chaque contrainte on associe la tension suivante :

$$T_i = \frac{\sum_{j=1}^N a_{ij}}{c_i}$$

Que l'on normalise par la suite de la manière suivante pour modérer l'impact des petits et des grands λ_i :

$$\hat{T}_i = \frac{T_i}{\sum_{k=1}^M T_k}$$

Un λ_i élevé signifie que la contrainte i est assez proche d'être violée et a donc un rôle important dans l'allocation des projets tandis qu'un λ_i faible signifie que la contrainte i est assez loin d'être violée et a donc un rôle secondaire dans l'allocation des projets.

Ainsi, une fois la solution initiale obtenue via la méthode de relaxation substitutive, l'heuristique de réparation se compose des étapes suivantes :

- On vérifie si il existe des contraintes violées
- Tant qu'il existe des contraintes violées, on calcule le rapport profit/coût pour les projets sélectionnés
- On trie les rapports profit/coût dans l'ordre croissant et on retire le projet avec le plus faible rapport de la solution.

- L'algorithme se termine quand une solution réalisable est trouvée

L'algorithme de l'heuristique de réparation a une complexité en $O(N \cdot M \log N)$.

3 Structures de voisinages

3.1 Construction des structures de voisinages

Un voisinage d'une solution x , noté $V(x)$, est défini comme l'ensemble des solutions issues de l'application d'une transformation t à x . Pour le problème de sac à dos multidimensionnel (0/1 MKP), nous avons implémenté plusieurs structures de voisinages adaptées à la nature combinatoire de la solution x , qui est représentée par un vecteur binaire.

3.2 Notion de structures multiples de voisinages

Dans le cadre de la recherche à voisinages variables, plusieurs structures de voisinage $\{V_1, V_2, \dots, V_{k_{\max}}\}$ sont définies, où chaque $V_k(x)$ représente un voisinage associé à un degré k de transformation. Ces voisinages permettent d'explorer différents paysages de recherche.

Pour obtenir plusieurs structures de voisinage, deux approches peuvent être considérées :

- Modifier la nature de la transformation t .
- Appliquer k transformations successives à une solution x , ce qui élargit progressivement le voisinage exploré.

3.3 Structures multiples de voisinages utilisées

1. Voisinage k -opt Dans le *voisinage k -opt*, k bits de la solution sont modifiés simultanément. Ce voisinage est basé sur le concept de distance de Hamming et explore toutes les combinaisons possibles de k modifications. Chaque modification consiste à inverser un bit x_i , c'est-à-dire passer de $x_i = 0$ à $x_i = 1$ ou inversement. Ce voisinage permet une exploration large. Considérons une solution initiale :

$$x = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}.$$

- **Exemple pour $k = 1$:**

Le voisinage $V_1(x)$ contient toutes les solutions obtenues en inversant un bit de x , soit :

$$V_1(x) = \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \right\}.$$

— **Exemple pour $k = 2$:**

Le voisinage $V_2(x)$ contient toutes les solutions obtenues en inversant deux bits simultanément. Cela donne :

$$V_2(x) = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right\}.$$

— **Exemple pour $k = 3$:**

Le voisinage $V_3(x)$ contient toutes les solutions obtenues en inversant trois bits simultanément. Ce voisinage est donné par :

$$V_3(x) = \left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right\}.$$

2. Voisinage par permutation (k -swap) Le *voisinage k -swap* consiste à échanger k paires d'éléments dans la solution. Cela signifie que pour chaque paire incluse/exclue ($x_i = 1$ et $x_j = 0$), on inverse leur état. Ce voisinage est particulièrement utile pour explorer des solutions proches, mais légèrement modifiées, tout en conservant une structure similaire à la solution initiale.

Considérons une solution initiale :

$$x = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}.$$

— **Exemple pour $k = 1$:**

Le voisinage $V_1(x)$ contient toutes les solutions obtenues en échangeant une seule paire d'éléments inclus/exclus ($0 \leftrightarrow 1$) :

$$V_1(x) = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \right\}.$$

— **Exemple pour $k = 2$:**

Le voisinage $V_2(x)$ contient toutes les solutions obtenues en échangeant deux paires d'éléments simul-

tanément :

$$V_2(x) = \left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

3. Voisinage par permutation basé sur les ratios profit/ressource Un autre type de voisinage exploite les caractéristiques spécifiques au problème du sac à dos multidimensionnel, notamment les ratios

$\frac{\text{profit}}{\text{consommation des ressources}}$. Plusieurs stratégies basées sur ces ratios ont été implémentées :

- **Remplacement des inclus les moins efficaces par les exclus les plus efficaces** : Ce voisinage identifie les k objets inclus ayant les plus faibles ratios $\frac{\text{profit}}{\text{ressource consommée}}$ ainsi que les k meilleurs exclus ayant les plus grands ratios. Il propose ensuite toutes les solutions dont une paire d'éléments inclus/exclus provenant de ces 2 groupes précédents a été échangé dans la solution courante. Ce voisinage améliore en théorie la solution en maximisant le profit pour une consommation fixe et permet donc intuitivement de proposer des solutions plus prometteuses. Cependant lorsque la solution courante est donnée par l'heuristique gloutonne définie précédemment, cette dernière est inutile car elle repose sur le même principe.
- **Remplacement inversé** : À l'inverse, ce voisinage remplace les k meilleurs inclus (ceux ayant les ratios les plus élevés) par les k pires exclus (les moins efficaces). Cette méthode est utilisée pour introduire une diversification dans la recherche et éviter les minima locaux.
- **Remplacement inversé version k -paires** : Il s'agit d'une extension du voisinage précédent qui permet de remplacer simultanément plusieurs paires d'éléments inclus/exclus, de 1 paire jusqu'à k paires, pour explorer des transformations plus complexes.

Exemple illustré

Problème

- Nombre d'objets : $N = 5$,
- Nombre de ressources : $M = 2$,
- Profits : $p = [10 \quad 15 \quad 25 \quad 20 \quad 30]$,
- Consommation des ressources :

$$\text{Consommation} = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix},$$

- Ressources disponibles : $b = [10 \quad 10]$.

Étape 1 : Calcul des ratios Les ratios sont calculés comme suit :

$$\text{Consommation totale} = \text{Somme des colonnes de la matrice consommation} = [3 \quad 5 \quad 7 \quad 9 \quad 11].$$

Les ratios $\frac{\text{profit}}{\text{consommation totale}}$ pour chaque objet sont :

$$\text{Ratios} = \left[\frac{10}{3}, \frac{15}{5}, \frac{25}{7}, \frac{20}{9}, \frac{30}{11} \right] \approx [3.33, 3.00, 3.57, 2.22, 2.73].$$

Étape 2 : Solution initiale donnée par l'heuristique gloutonne L'heuristique gloutonne sélectionne les objets en fonction des ratios décroissants, tout en respectant les contraintes de ressources.

- Objets triés par ratios décroissants : $\{3, 1, 2, 5, 4\}$,
- Objets sélectionnés : $\{3, 1, 2\}$ (leurs consommations respectent les contraintes),
- Solution initiale :

$$x = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \text{Profit total} = 10 + 15 + 25 = 50.$$

Exemple des voisinages $V_k(x)$ pour le remplacement des inclus les moins efficaces par les exclus les plus efficaces)

$$\text{Ratios inclus} = \left[\frac{10}{3}, \frac{15}{5}, \frac{25}{7} \right] \approx [3.33, 3.00, 3.57], \quad \text{Ratios exclus} = \left[\frac{20}{9}, \frac{30}{11} \right] \approx [2.22, 2.73].$$

Les objets inclus sont triés par ratio croissant d'efficacité : $\{2, 1, 3\}$. Les objets exclus sont triés par ratio décroissant d'efficacité : $\{5, 4\}$.

- **Exemple pour $k = 1$:**

On remplace un objet inclus parmi $k = 1$ par un objet exclu parmi $k = 1$.

$$V_1(x) = \left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix} \right\}.$$

- **Exemple pour $k = 2$:**

On remplace un objet inclu parmi $k = 2$ par un objet exclu parmi $k = 2$.

$$V_2(x) = \left\{ \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

- **Exemple pour $k = 3$:**

On remplace un objet inclu parmi $k = 3$ par un objet exclu parmi $k = 3$.

$$V_3(x) = \left\{ \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

Exemple des voisinages $V_k(x)$ pour le remplacement des inclus les plus efficaces par les exclus les moins efficaces)

$$\text{Ratios inclus} = \left[\frac{10}{3}, \frac{15}{5}, \frac{25}{7} \right] \approx [3.33, 3.00, 3.57], \quad \text{Ratios exclus} = \left[\frac{20}{9}, \frac{30}{11} \right] \approx [2.22, 2.73].$$

Les objets inclus sont triés par ratio décroissant d'efficacité : $\{3, 1, 2\}$. Les objets exclus sont triés par ratio croissant d'efficacité : $\{4, 5\}$.

— **Exemple pour $k = 1$:**

On remplace un objet inclu parmi $k = 1$ par un objet exclu parmi $k = 1$.

$$V_1(x) = \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

— **Exemple pour $k = 2$:**

On remplace un objet inclu parmi $k = 2$ par un objet exclu parmi $k = 2$.

$$V_2(x) = \left\{ \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

— **Exemple pour $k = 3$:**

On remplace un objet inclu parmi $k = 3$ par un objet exclu parmi $k = 3$.

$$V_3(x) = \left\{ \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

Exemple des voisinages $V_k(x)$ pour le remplacement des inclus les plus efficaces par les exclus les moins efficaces, version k-pairs)

$$\text{Ratios inclus} = \left[\frac{10}{3}, \frac{15}{5}, \frac{25}{7} \right] \approx [3.33, 3.00, 3.57], \quad \text{Ratios exclus} = \left[\frac{20}{9}, \frac{30}{11} \right] \approx [2.22, 2.73].$$

Les objets inclus sont triés par ratio décroissant d'efficacité : $\{3, 1, 2\}$. Les objets exclus sont triés par ratio croissant d'efficacité : $\{4, 5\}$.

— **Exemple pour $k = 1$:**

On remplace $k = 1$ objet inclus parmi $k = 1$ par un objet exclu parmi $k = 1$.

$$V_1(x) = \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \right\}.$$

— **Exemple pour $k = 2$:**

On remplace 1 puis 2 objets inclus parmi $k = 2$ par respectivement 1 puis 2 objets exclus parmi $k = 2$.

$$V_2(x) = \left\{ \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

— **Exemple pour $k = 3$:**

On remplace 1 puis 2 puis 3 objets inclus parmi $k = 3$ par respectivement 1 puis 2 puis 3 objets exclus parmi $k = 3$.

$$V_3(x) = \left\{ \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

Les voisinages tels que k -opt est plus général et permet une exploration large tandis que les 2 voisinages basés sur les échanges sont moins grands mais contiennent les solutions proches les plus prometteuses. Le dernier voisinage basé sur les ratios profit/ressources exploite les spécificités du problème pour guider encore plus la recherche vers les solutions les plus prometteuses.

4 Heuristique d'amélioration locale

L'heuristique d'amélioration locale est une méthode simple et efficace pour atteindre un optimum local rapidement. Cependant elle n'offre pas la possibilité de sortir de ces optima locaux : il faudra donc utiliser une métaheuristique par la suite.

4.1 Pseudo-code

Algorithm 1: Algorithme d'une heuristique d'amélioration locale
Input: Solution initiale x_0 faisable (obtenue par l'heuristique gloutonne dans notre cas)
Output: Solution optimisée $x_{\max}$ correspondant à un optimum local
1 Initialisation :
2 Poser $x_{\max} \leftarrow x_0$ // Meilleure solution rencontrée
3 $Fin \leftarrow \text{faux}$ // Indicateur d'arrêt
4 while $\neg Fin$ do
5 $Fin \leftarrow \text{vrai};$
6 $x \leftarrow x_{\max}$ // Solution courante
7 Construire le voisinage $V(x)$ de x ;
8 for $x' \in V(x)$ do
9 if $f(x') > f(x_{\max})$ then
10 $x_{\max} \leftarrow x';$
11 end
12 end
13 if $x \neq x_{\max}$ then
14 $Fin \leftarrow \text{faux}$ // Une solution améliorante existe
15 end
16 end
17 return $x_{\max}$ // Retourner la meilleure solution trouvée

4.2 Implémentation sur Python

Pour l'implémentation nous avons utilisée des méthodes pour améliorer le temps de calcul. Au lieu de construire explicitement les voisins de la solution courante, l'algorithme génère les **indices des bits à inverser** qui correspondent aux modifications nécessaires pour passer à un voisin. Cela réduit le coût en mémoire et en calcul.

Soit $\mathbf{x}$ la solution courante, et $\mathcal{N}(\mathbf{x})$ son voisinage. Pour un paramètre k , la fonction `generate_neighbors` produit des indices $I \subseteq \{1, \dots, N\}$ de taille k définissant les bits de $\mathbf{x}$ à inverser.

$$\mathcal{N}(\mathbf{x}) = \{\mathbf{x}' \mid \mathbf{x}'[i] = 1 - \mathbf{x}[i], \forall i \in I, I \in \text{generate_neighbors}(\mathbf{x})\}.$$

Pour évaluer et parcourir efficacement le voisinage :

- Une **matrice de deltas** $\Delta \in \{-1, 0, 1\}^{|\mathcal{N}(\mathbf{x})| \times N}$ est créée, où chaque ligne encode les modifications de $\mathbf{x}$ pour un voisin donné.
- Les changements dans la consommation des ressources sont calculés en une seule opération matricielle :

$$\Delta \mathbf{R} = \mathbf{R} \cdot \Delta^T,$$

où $\Delta \mathbf{R} \in \mathbb{R}^{M \times |\mathcal{N}(\mathbf{x})|}$ représente les deltas de consommation des ressources pour tous les voisins.

- La faisabilité des voisins est vérifiée en évaluant :

$$\mathbf{R}\mathbf{x} + \Delta \mathbf{R} \leq \mathbf{A}.$$

Cette étape est réalisée de manière vectorisée pour éviter une boucle sur chaque voisin.

- Les variations de profit pour tous les voisins faisables sont calculées comme suit :

$$\Delta P = \mathbf{p} \cdot \Delta^T,$$

où $\mathbf{p} \in \mathbb{R}^N$ est le vecteur des profits des projets. Le meilleur voisin est sélectionné en maximisant :

$$P(\mathbf{x}) + \Delta P.$$

Toutes ces méthodes permettent d'utiliser la structure du problème pour n'avoir à faire que les calculs nécessaire, éviter de refaire les calculs en entier à chaque fois et ainsi économiser du temps.

Nous avons testé les différents voisinages proposés. Bien que le voisinage basé sur la distance de Hamming était le plus simple et celui qui utilisait le moins la structure du problème, c'était celui qui proposait la meilleure diversité et donc les meilleures performances. Nous sommes donc restés sur le voisinage 3 – *opt*.

5 Métaheuristiques

5.1 Métaheuristique de recherche à voisinages variables

Pour poursuivre la recherche au delà des optima locaux, nous utilisons une métaheuristique de recherche par voisinages variables (VNS). Cette dernière introduit des structures multiples de voisinages ainsi que des perturbations aléatoires pour sortir des optima locaux.

Algorithm 2: Recherche à voisinages variables (VNS)

Data: Une solution initiale x_0 , un temps maximal $tps_{\max}$, et un nombre maximum de voisinages

$k_{\max}$.

Result: Une solution améliorée x .

```
1  $tps \leftarrow 0$ ;  $x \leftarrow x_0$ ;
2 while  $tps \leq tps_{\max}$  do
3    $k \leftarrow 1$ ;
4   while  $k \leq k_{\max}$  do
5     Étape de perturbation
6     Générer  $x'$  aléatoirement dans  $V_k(x)$ ;
7     Étape de recherche locale
8     Appliquer une heuristique de recherche locale à partir de  $x'$ ;
9     Soit  $x''$ , l'optimum local ainsi obtenu;
10    Étape de déplacement
11    if  $f(x'') < f(x)$  then
12       $x \leftarrow x''$ ;
13       $k \leftarrow 1$ ;
14    else
15       $k \leftarrow k + 1$ ;
16    end
17  end
18   $tps \leftarrow \text{Cputime}()$ ;
19 end
20 return  $x$ 
```

Pour la solution initiale, nous sommes partis de la solution gloutonne car elle était très performante et nous permettait de commencer avec une solution assez bonne et donc gagner en temps de calcul. Pour le voisinage, nous sommes restés sur 3 – *opt*. Pour l'étape de recherche locale, nous avons utilisé la méthode de montée précédente. Cette métaheuristique était très performante. Pour aller plus loin, nous avons testé d'autres métaheurstiques pour les comparer avec la performance de la VNS.

5.2 Algorithme génétique

Les algorithmes génétiques (AG) sont des métaheurstiques inspirées du processus de sélection naturelle. Ils manipulent une population de solutions candidates pour explorer l'espace de recherche et converger vers des solutions optimales ou proches de l'optimum. Voici les principales étapes d'un algorithme génétique :

Nous avons également implémenté un algorithme génétique afin de travailler sur un vaste espace de recherche et ainsi éviter de rester coincé dans un même maximum local

5.2.1 Étapes de l'algorithme génétique

1. Initialisation :

- On génère une population initiale de N_p individus. Chaque individu représente une solution candidate au problème, codée sous la forme d'un vecteur binaire ou réel. Nous avons fait le choix de générer la population en créant N_p solutions à partir de l'heuristique de réparation utilisant elle-même une solution aléatoire comme solution initiale.
- Calculer la valeur de la fonction objectif pour chaque individu.

2. Évaluation :

- On assigne une fitness à chaque individu en fonction de la qualité de la solution qu'il représente (par exemple, la valeur de la fonction objectif).

3. Sélection :

- On sélectionne les individus pour reproduction à l'aide d'un mécanisme probabiliste. Nous avons la sélection par tournoi qui consiste à prendre au hasard un sous-groupe d'individus et sélectionner le meilleur parmi eux.

4. Croisement :

- On combine les gènes de deux parents pour produire un ou plusieurs descendants. Nous avons utilisé un croisement uniforme avec une probabilité de croisement de chaque gène fixée à 0.5 and une probabilité de croisement de chaque individu fixée à 0.7. Chaque gène du descendant est choisi aléatoirement parmi les gènes correspondants des deux parents.

5. Mutation :

- On modifie aléatoirement un ou plusieurs gènes d'un individu avec une probabilité P_{mutation} fixée à 0.2 à l'échelle du gène comme de l'individu. Cela permet d'introduire de la diversité dans la population.

6. Remplacement :

- On met à jour la population en remplaçant tout ou partie des anciens individus par les nouveaux descendants.

7. Critère d'arrêt :

- Répéter les étapes de sélection, croisement et mutation jusqu'à ce qu'un critère d'arrêt soit atteint (par exemple, un nombre maximal de générations ou une convergence de la fitness).

5.2.2 Pseudo-code de l'algorithme génétique

Algorithm 3: Pseudo-code de l'algorithme génétique

Data: Population initiale P_0 , taille de la population N_p , probabilité de croisement P_c , probabilité de mutation P_m , nombre maximal de générations $G_{\max}$.

Result: Meilleure solution x_{best} .

```
1 Initialiser la population  $P_0$  aléatoirement;
2 Évaluer la fitness de chaque individu dans  $P_0$ ;
3  $g \leftarrow 0$ ;                                /* Initialisation du compteur de générations */
4 while  $g < G_{\max}$  do
5     Sélection : Sélectionner  $N_p$  individus à reproduire;
6     Croisement : Appliquer le croisement avec une probabilité  $P_c$ ;
7     Mutation : Appliquer la mutation avec une probabilité  $P_m$ ;
8     Évaluation : Évaluer la fitness des descendants;
9     Remplacement : Mettre à jour la population  $P_{g+1}$  avec les descendants;
10     $g \leftarrow g + 1$ ;
11 end
12 Retourner la meilleure solution  $x_{\text{best}}$  dans la population;
```

Pour implémenter l'algorithme génétique, nous utilisons la librairie Python DEAP.

5.3 Recuit simulé

Le recuit simulé ou simulated annealing (SA) en anglais est un algorithme qui s'inspire de la métallurgie.

5.3.1 Pseudo-code de l'algorithme génétique

Algorithm 4: Algorithme de base du recuit simulé pour un problème de minimisation

Data: $x_{\min}$: meilleure solution rencontrée, x : solution courante

n_{biter} : nombre d'itérations

μ : coefficient de diminution de la température courante T

Result: Une solution $x_{\min}$ minimisant la fonction f .

```

1 Trouver  $x_0 \in X$ ; ; /* Déterminer une solution initiale */
2 Choisir un nombre d'itérations  $iter_{\max}$  à partir duquel  $T$  est diminuée;
3 Choisir  $T_0 > 0$  et  $0 < \mu < 1$ ;
4  $x_{\min} \leftarrow x_0$ ;  $x \leftarrow x_0$ ;  $T \leftarrow T_0$ ;
5 while Critère d'arrêt non satisfait do
6      $k \leftarrow 0$ ; ; /*  $k$  est un compteur d'itérations */
7     for  $n_{\text{biter}}$  allant de 1 à  $iter_{\max}$  do
8          $k \leftarrow k + 1$ ;
9         Construire un voisinage  $V(x)$  de  $x$ ;
10        Tirer au sort  $x'$  dans  $V(x)$ ;
11         $\Delta f \leftarrow f(x') - f(x)$ ;
12        if  $\Delta f \leq 0$  then
13             $x \leftarrow x'$ ;
14            if  $f(x) < f(x_{\min})$  then
15                 $x_{\min} \leftarrow x$ ;
16            end
17        else
18            Tirer au sort un nombre  $q \in [0, 1]$ ;
19            if  $q \leq e^{-\frac{\Delta f}{T}}$  then
20                 $x \leftarrow x'$ ;
21            end
22        end
23    end
24     $T \leftarrow \mu \times T$ ;
25 end
26 return  $x_{\min}$ 

```

Pour la solution initiale, nous sommes partis de la solution gloutonne car elle était très performante et nous permettait de commencer avec une solution assez bonne et donc gagner en temps de calcul.

Détermination de la valeur initiale de la température : T_0

On fixe une probabilité d'acceptation de solution dégradante, notée π , à atteindre au démarrage. Tester un certain nombre de transformations et évaluer la variation de coût moyenne $\Delta f > 0$ des solutions coûteuses engendrées. $T_0 = \frac{-\Delta f}{\ln \pi}$.

Nous avons utilisé les paramètres suivants qui nous donnaient de bonnes performances après des tests empiriques.

- un modèle de décroissance linéaire avec comme coefficient de décroissance $\mu = 0.95$ car cela nous donnait de bonnes performances.
- $iter_{max} = 100$.
- comme voisinage 3 – *opt*.

Les performances de cet algorithme étaient légèrement inférieures aux 2 métaheuristiques précédentes.

5.4 Algorithme hybride SA-IGA

L'algorithme hybride SA-IGA combine les forces du *Recuit Simulé* (*Simulated Annealing*, SA) et de l'*Algorithme Génétique* (*Improved Genetic Algorithm*, IGA). L'idée principale est d'exploiter la capacité du SA à s'échapper des minima locaux grâce à un mécanisme probabiliste tout en utilisant les mécanismes de sélection et de croisement de GA pour explorer efficacement l'espace de recherche. Sur le papier il semble très performant car combine des avantages. Cependant, dans les faits, son coût computationnel est trop important du fait qu'il combine deux métaheuristiques demandantes. Nous avons effectué des tests mais ceux-ci n'étant pas concluants nous l'avons omis dans ce rapport.

Source : A Comparative Study of Meta-Heuristic Optimization Algorithms for 0-1 Knapsack Problem : Some Initial Results (ABSALOM E. EZUGWU, VEROSHA PILLAY, DIVYAN HIRASEN2, KERSHEN SIVANARAIN, AND MELVIN GOVENDER).

6 Exploitation des résultats

6.1 Structure des instances and métriques d'évaluation choisies

6.1.1 Description des instances de test

Afin de tester nos différentes heuristiques et métaheuristiques, nous disposons de 9 fichiers .txt contenant chacune 30 instances présentant les mêmes nombres de projets et de ressources mais des profits, coûts et capacités différents. La figure 1 donne le détail du nombre de projets N et du nombre de ressources M pour chacun des 9 fichiers de test.

Ces fichiers nous permettent d'évaluer la sensibilité de nos heuristiques et métaheuristiques face à des variations de N et M.

Fichiers	Nombre projets N	Nombres de ressources M
mknapcb1	100	5
mknapcb2	250	5
mknapcb3	500	5
mknapcb4	100	10
mknapcb5	250	10
mknapcb6	500	10
mknapcb7	100	30
mknapcb8	250	30
mknapcb9	500	30

FIGURE 1 – Paramètres N et M selon les fichiers de test

6.1.2 Métriques d'évaluation

Pour mesurer les performances de nos tests, nous avons retenu comme métriques l'écart entre le profit optimal et le profit trouvé défini de la manière suivante :

$$\text{Ecart} = \frac{|P^* - P_{\text{heuristique}}|}{P^*}$$

Nous avons également retenu le temps de résolution. Nous avons par ailleurs choisi de ne pas considérer les solutions mettant plus de 10 minutes à être trouvées. Pour les deux métriques, nous avons calculé des statistiques élémentaires sur les instances telles que les valeurs maximum et minimum, la moyenne et l'écart-type.

6.2 Performances des heuristiques préliminaires

6.2.1 Performances globales

6.2.2 Sensibilité par rapport à N et M

Les heuristiques gloutonne et de réparation ont été testées pour toutes les instances disponibles à savoir 270 instances. Les statistiques globales sont données en figure 2.

Statistiques	Ecart [Glouton]	Ecart [Réparation]	Temps [Glouton] (en s)	Temps [Réparation] (en s)
Valeur min	0.28%	0.53%	0.002	0.001
Valeur max	16.60%	26.15%	0.185	0.200
Moyenne	3.43%	5.47%	0.026	0.031
Ecart-type	2.55%	4.16%	0.026	0.038

FIGURE 2 – Statistiques pour les heuristiques gloutonne et de réparation

Nous remarquons les deux heuristiques obtiennent en moyenne des profits relativement proches des profits optimaux. L'heuristique gloutonne est la plus performante avec seulement 3.43% d'écart en moyenne. Les heuristiques sont résolues en moins d'une seconde quelque soit l'instance choisie.

6.2.3 Sensibilité par rapport à N et M

Nous avons également tracé l'écart moyen de nos deux heuristiques en fonction du nombre de projets N (figure 3).

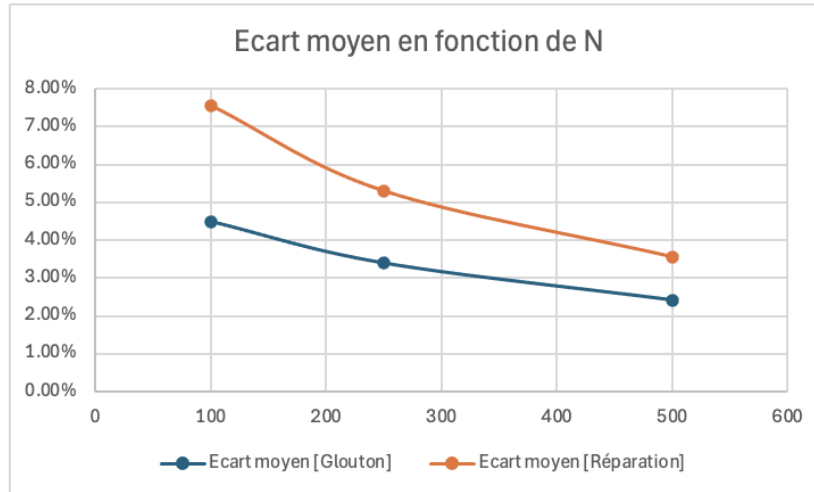


FIGURE 3 – Ecart moyen par rapport au nombre de projets N

Nous remarquons les deux heuristiques ont tendance à avoir une meilleure fiabilité quand le nombre de projets augmente. Nous avons aussi tracé l'écart moyen en fonction du nombre de ressources M. (figure 4).

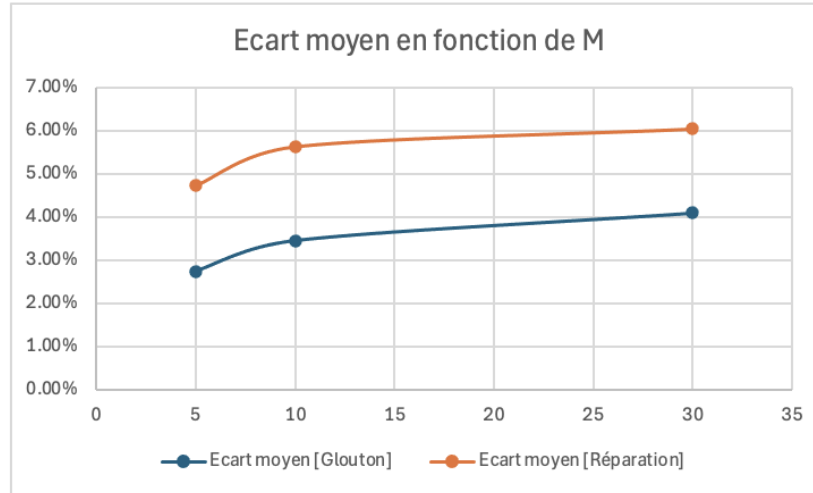


FIGURE 4 – Ecart moyen par rapport au nombre de ressources M

On observe ainsi que l'augmentation du nombre de ressources et donc de contraintes rend les deux heuristiques moins fiables.

6.3 Performances de l'heuristique d'amélioration locale

6.3.1 Performances avec un voisinage 3-opt

Pour l'heuristique d'amélioration locale avec le voisinage 3-opt, nous avons essayé de lancer 5 instances pour chacun des 9 fichiers par contrainte de temps. Nous avons ainsi obtenu des solutions avec des temps de résolution inférieurs à 10 minutes pour les fichiers pour tous les fichiers sauf les 3, 6, 8 et 9. Les performances globales sont données en figure 5.

Statistiques	Ecart	Temps (en s)
Valeur min	0.31%	7.16
Valeur max	5.23%	753.74
Moyenne	2.06%	143.83
Ecart-type	1.37%	186.77

FIGURE 5 – Performances globales pour l'heuristique d'amélioration locale avec voisinage 3-opt

Nous remarquons cette heuristique présente des écarts moyens meilleurs que les heuristiques préliminaires (glouton et réparation). Ceci montre ainsi que le voisinage 3-opt est pertinent car il permet d'améliorer localement la solution obtenue via l'heuristique gloutonne. En revanche, nous observons notamment que le temps de résolution augmente sensiblement quand le nombre de projets N passe de 100 à 250 comme le montre la figure 6.

Statistiques	Ecart (N=100)	Ecart (N=250)	Temps (N=100) (en s)	Temps (N=250) (en s)
Valeur min	0.69%	0.31%	7.16	214.92
Valeur max	5.23%	1.62%	26.17	753.74
Moyenne	2.75%	1.03%	13.71	339.02
Ecart-type	1.35%	0.41%	5.04	153.87

FIGURE 6 – Performances en fonction de N

Cette augmentation résulte du fait qu'il y a beaucoup plus de voisins à construire quand on passe de 100 à 250 projets ce qui est très coûteux. En revanche nous remarquons que l'écart moyen est plus pour N=100 que N=250 ce qui s'explique par le fait que le voisinage est plus grand donc plus propice pour trouver une solution locale améliorante. Par ailleurs, l'heuristique s'avère moins fiable quand le nombre de ressources, donc de contraintes augmente comme le montre la figure 7.

M	Ecart moyen	Temps moyen
5	1.38%	11.19
10	2.77%	11.42
30	4.09%	18.52

FIGURE 7 – Performances en fonction de M

6.4 Performances de l'algorithme génétique

Pour l'algorithme génétique, nous avons choisi de lancer 5 instances pour chacun des 9 fichiers. Lors de nos tests, nous avons conclu que les paramètres discriminants pour assurer de bonnes performances était la taille de la population initiale N_p et le nombre de génération N_g . Nous avons ainsi déterminé de manière empirique ces paramètres. Pour N_p , nous avons déterminé la valeur permettant d'avoir les meilleures performances dans un temps inférieur à 10 minutes. Pour N_g , nous avons pris un nombre de génération raisonnable de manière à avoir une solution stabilisée. Les paramètres choisis pour chaque fichier sont exposés en figure 8.

Fichier	N_p	N_g
knapcb1	400	100
knapcb2	300	100
knapcb4	700	80
knapcb7	800	80
knapcb5	250	100
knapcb8	250	100
knapcb3	300	150
knapcb6	300	150
knapcb9	200	150

FIGURE 8 – Paramètres retenus pour chaque fichier

Les performances globales de l'algorithme génétique sont présentées en figure 9.

Statistiques	Ecart	Temps
Valeur min	0.73%	8.91
Valeur max	6.10%	424.20
Moyenne	2.49%	86.18
Ecart-type	0.96%	87.13

FIGURE 9 – Performances globales de l'algorithme génétique

Nous remarquons que les performances en termes d'écarts de l'algorithme génétique sont en moyenne similaires à l'heuristique d'amélioration locale (autour de 2% d'écart) à la différence que l'algorithme génétique permet de résoudre les instances de tous les fichiers dans temps inférieur à 10 minutes.

7 Conclusion

Pour conclure, ce travail a permis d’explorer différentes approches heuristiques et métaheuristiques pour résoudre le problème du sac à dos multidimensionnel. Les résultats obtenus montrent une complémentarité entre les différentes stratégies étudiées.

Les heuristiques préliminaires, bien que rapides et simples à implémenter, permettent d’obtenir des solutions initiales fiables, en particulier l’heuristique gloutonne qui présente un écart moyen de seulement 3,43% avec les solutions optimales. Toutefois, leur performance diminue lorsque le nombre de ressources augmente, soulignant la nécessité d’utiliser des méthodes plus sophistiquées pour traiter des instances complexes.

L’heuristique d’amélioration locale, basée sur un voisinage 3-opt, a démontré sa pertinence en améliorant significativement les solutions initiales. Cependant, son coût computationnel devient prohibitif pour des instances de grande taille, notamment lorsque le nombre de projets augmente.

Parmi les métaheuristiques, l’algorithme génétique a offert une solution robuste et adaptable. Grâce à sa capacité à explorer un large espace de recherche tout en évitant les minima locaux, il a permis de résoudre toutes les instances dans des temps raisonnables. De plus, ses performances, en termes d’écarts avec les optima, sont comparables à celles de l’heuristique d’amélioration locale tout en offrant une meilleure scalabilité.

Enfin, bien que le recuit simulé ait montré des résultats corrects, ses performances étaient légèrement inférieures aux deux autres métaheuristiques testées. De plus, l’algorithme hybride SA-IGA, bien que prometteur sur le papier, s’est avéré trop coûteux en temps de calcul pour les instances étudiées.

En perspective, il serait intéressant d’explorer des combinaisons hybrides plus légères ou des algorithmes basés sur l’apprentissage automatique pour guider la recherche dans les voisinages. De plus, l’intégration de méthodes de relaxation avancées pourrait permettre une meilleure initialisation des solutions pour des instances encore plus complexes.

En somme, ce rapport illustre l’importance de choisir et d’adapter les méthodologies selon les caractéristiques spécifiques du problème, tout en tenant compte des contraintes computationnelles et des performances attendues.